Automate the Boring Stuff with Python, 3rd Edition

Practical Programming for Total Beginners

by Al Sweigart

Errata updated to print 3

Page	Error	Correction	Print corrected

xxxiii	`if typed_password == secret_password:`
	`  print('Access granted')`
	`  if typed_password == '12345':`
		`if typed_password == secret_password:`
		`  print('Access granted')`
		`  if secret_password == '12345':`
			Print 2

xxxvii	LLMs will not replace most human jobs (though this won't prevent your manager from thinking **they can** and laying you off anyway).	LLMs will not replace most human jobs (though this won't prevent your manager from thinking **so** and laying you off anyway).	Print 3
xlii	*Figure 2: The **Google** results for an error message can be very helpful.*	*Figure 2: The **DuckDuckGo** results for an error message can be very helpful.*	Print 3
xliii	You'll explore how to compile Python scripts into executable programs on Windows, macOS, and **Linus**; and create command line programs and run them.	You'll explore how to compile Python scripts into executable programs on Windows, macOS, and **Linux**; and create command line programs and run them.	Print 2
29	`SyntaxError: can't assign to False`	`SyntaxError: cannot assign to False`	Print 3
66	`else:` `  print('Nope. The number was ' + str(secret_number))`	`else:` `  print('The number was ' + str(secret_number))`	Print 2
89	`  if indent_increasing`	`  if indent_increasing:`	Print 2
92	By having **exponentially** longer strings, we create the top half of the spike.	By having **quadratically** longer strings, we create the top half of the spike.	Print 3
94	To make the output more compact, the `print()` calls that print the numbers should have **a `sep=' '`** named parameter to print all values on one line.	To make the output more compact, the `print()` calls that print the numbers should have **an `end=' '`** named parameter to print all values on one line.	Print 2

Page	Error	Correction	Print corrected

102	`>>> logging.debug('Some minor code and debugging details.')`
	`2035-05-18 19:04:26,901 - DEBUG - Some debugging details.`
	`>>> logging .info('An event happened.')`
	`2035-05-18 19:04:35,569 - INFO - The logging module is working.`
	`>>> logging.warning('Something could go wrong.')`
	`2035-05-18 19:04:56,843 - WARNING - An error messaged is about to be logged.`
		`>>> logging.debug('Some debugging details.')`
		`2035-05-18 19:04:26,901 - DEBUG - Some debugging details.`
		`>>> logging .info('An event happened.')`
		`2035-05-18 19:04:35,569 - INFO - An event happened.`
		`>>> logging.warning('Something could go wrong.')`
		`2035-05-18 19:04:56,843 - WARNING - Something could go wrong.`
			Print 3

111	`>> spam = ['cat', 'bat', 'rat', 'elephant']`	`>>> spam = ['cat', 'bat', 'rat', 'elephant']`	Print 3
118	You can consider `random.choice(`*`some_list`*`)` to be a shorter form of *`some_list`*`[random.randint(0, len(`*`some_list`*`) - 1]`.	You can consider `random.choice(`*`some_list`*`)` to be a shorter form of *`some_list`*`[random.randint(0, len(`*`some_list`*`) - 1)]`.	Print 3
125	Selecting a random item from a list is common enough that Python has the `random.choice(messages)` function that does the same thing as `random.rand int(0, len(messages) - 1)`.	Selecting a random item from a list is common enough that Python has the `random.choice(messages)` function that does the same thing as `messages[random.rand int(0, len(messages) - 1)]`.	Print 3
137	`for experiment_number in range(10000):  # Run 100,000 experiments total.`	`for experiment_number in range(10000):  # Run 10,000 experiments total.`	Print 3
145	The first time we call `setdefault()`, the dictionary in spam changes to `{'color': 'black', 'age': 5, 'name': 'Pooka'}`.	The first time we call `setdefault()`, the dictionary in spam changes to `{'name': 'Pooka'} 'age': 5, 'color': 'black'`.	Print 2
151	Two nested for loops will loop over all 64 squares on the board, starting with the upper-left square and going **right** to **left**, then top to bottom.	Two nested for loops will loop over all 64 squares on the board, starting with the upper-left square and going **left** to **right**, then top to bottom.	Print 3
178–179	*As the book's convention is to use camel_case rather than snakeCase, the following variable names should be updated:* • *`pigLatin` should be `pig_latin`* • *`prefixNonLetters` should be `prefix_non_letters`* • *`suffixNonLetters` should be `suffix_non_letters`* • *`wasUpper` should be `was_upper`* • *`wasTitle` should be `was_title`* • *`prefixConsonants` should be `prefix_consonants`*		Print 2
178	`suffix_non_letters += word[-1] + suffix_non_letters`	`suffix_non_letters = word[-1] + suffix_non_letters`	Print 3

Page	Error	Correction	Print corrected
194	`['12 drummers', '11 pipers', '10 lords', '9 ladies', '8 maids', '7 swans', '` `6 geese', '5 rings', '4 birds', '3 hens', '2 doves', '1 partridge']`	`['12 drummers', '11 pipers', '10 lords', '9 ladies', '8 maids', '7 swans',` `'6 geese', '5 rings', '4 birds', '3 hens', '2 doves', '1 partridge']`	Print 3
208	`❶ for groups in email_re.findall(text):` `  matches.append(groups)`	`❶ for groups in email_re.findall(text):` `  matches.append(groups[0])`	Print 2
221	`WindowsPath('C:/Users/Al/AppData/Local/Programs/Python/Python313')'`	`WindowsPath('C:/Users/Al/AppData/Local/Programs/Python/Python313')`	Print 2
223	`WindowsPath('C:/Users/Al/AppData/Local/Programs/Python/Python312')`	`WindowsPath('C:/Users/Al/AppData/Local/Programs/Python/Python313')`	Print 3

226	`st_nlink=2, st_uid=0, st_gid=0, st_size=27648, st_atime=1678984560,`
	`st_mtime=1575709787, st_ctime=1575709787)`
	`>>> calc_file.stat().st_size`
	`27648`
	`>>> calc_file.stat().st_mtime`
	`1712627129.0906117`
		`st_nlink=2, st_uid=0, st_gid=0, st_size=27648, st_atime=1712627129,`
		`st_mtime=1712627129, st_ctime=1712627129)`
		`>>> calc_file.stat().st_size`
		`27648`
		`>>> calc_file.stat().st_mtime`
		`1712627129`
			Print 3

239	`1. What is the capital of West Virginia?`
	`  A. Hartford`
	`  B. Santa Fe`
	`  C. Harrisburg`
	`  D. Charleston`
	
	`2. What is the capital of Colorado?`
	`  A. Raleigh`
	`  B. Harrisburg`
	`  C. Denver`
	`  D. Lincoln`
		`1. Capital of West Virginia:`
		`  A. Hartford`
		`  B. Santa Fe`
		`  C. Harrisburg`
		`  D. Charleston`
		
		`2. Capital of Colorado:`
		`  A. Raleigh`
		`  B. Harrisburg`
		`  C. Denver`
		`  D. Lincoln`
			Print 3

251	For instance, if you replaced the call at ❶ with `example_zip.extractall('C:\spam)`, the code would extract the files from *example.zip* into a newly created *C:\spam* folder.	For instance, if you replaced the call at ❶ with `example_zip.extractall('C:\spam')`, the code would extract the files from *example.zip* into a newly created *C:\spam* folder.	Print 3

Page	Error	Correction	Print corrected

254	`Creating spam_1.zip...`
	`Adding files in spam...`
	`Adding file file1.txt...`
	`Done.`
		`Creating spam_1.zip...`
		`Adding files in folder spam...`
		`Adding file file1.txt...`
		`Done.`
			Print 3

267	`list(sys.version_info)` returns `[3, 13, 1 'final', 0]` on my laptop.	`list(sys.version_info)` returns `[3, 13, 1, 'final', 0]` on my laptop.	Print 3
269	Python scripts can access the command line **scripts** given to the Python interpreter from the `sys.argv` list.	Python scripts can access the command line **arguments** given to the Python interpreter from the `sys.argv` list.	Print 3
272	The `os.system()` call**, which you'll learn more about in Chapter 17,** runs the `cls` program (on Windows) or the `clear` program (on macOS and Linux).	The `os.system()` runs the `cls` program (on Windows) or the `clear` program (on macOS and Linux).	Print 3

284	`[Desktop Entry]`
	`Name=Clipboard Recorder`
	`Exec=gnome-terminal -- /home/al/cliprec`
	`Type=Application`
		`[Desktop Entry]`
		`Name=Clipboard Recorder`
		`Exec=gnome-terminal -- /home/al/Scripts/cliprec`
		`Type=Application`
			Print 3

308–309, 589, 607	**`gettext()`**	**`get_text()`**	Print 2
311	`# Retrieve top search result links.` `soup = bs4.BeautifulSoup(res.text, 'parser.html')`	`# Retrieve top search result links.` `soup = bs4.BeautifulSoup(res.text, 'html.parser')`	Print 2
319	**`Browser.quit()`** Clicks the Close Window button	**`browser.quit()`** Clicks the Close Window button	Print 2
354	The column and row integer of the top-left cell of the rectangular selection of cells containing your chart data: the first integer in the tuple is the **row**, and the second is the **column**.	The column and row integer of the top-left cell of the rectangular selection of cells containing your chart data: the first integer in the tuple is the **column**, and the second is the **row**.	Print 3
355	For the following questions, imagine you have a `Workbook` object in the variable `wb`, a `Worksheet` object in `sheet`**, and a `Sheet` object in `sheet`**.	For the following questions, imagine you have a `Workbook` object in the variable `wb` **and** a `Worksheet` object in `sheet`**.**	Print 3
356	10. If you needed to get the string name for **row** `14`, what function would you need to call?	10. If you needed to get the string name for **column** `14`, what function would you need to call?	Print 3
389	There are **six** data types in SQLite: **`NULL`** Analogous to Python's `None` **`INT` or `INTEGER`** Analogous to Python's `int` type	There are **five** data types in SQLite: **`NULL`** Analogous to Python's `None` **`INTEGER`** Analogous to Python's `int` type	Print 3

Page	Error	Correction	Print corrected
399	`[('idx_birthdate',) ('idx_name',)]`	`[('idx_birthdate',), ('idx_name',)]`	Print 3
400	If you forgot to include the `WHERE` clause, the updates would apply to every row in the **table. and** suddenly all of your cats would have orange and white fur!	If you forgot to include the `WHERE` clause, the updates would apply to every row in the **table and** suddenly all of your cats would have orange and white fur!	Print 3
405	The integer values in this column **matches** the `rowid` values of the rows in the `cats` table.	The integer values in this column **match** the `rowid` values of the rows in the `cats` table.	Print 3

423	`for filename in os.listdir('.'):`
	`  if filename.endswith('.pdf'):`
	`  pdf_files.append(filename)`
		`for filename in os.listdir('.'):`
		`  if filename.endswith('.pdf'):`
		`  pdf_filenames.append(filename)`
			Print 3

425	`>>> doc.paragraphs[1].text` `'A plain paragraph with some bold text and some italic'`	`>>> doc.paragraphs[1].text` `'A plain paragraph with some bold and some italic'`	Print 3
426	Here, the first text attribute contains `'DocumentTitle'`, and the second contains `'A plain paragraph with some bold text and some italic'`.	Here, the first text attribute contains `'Document Title'`, and the second contains `'A plain paragraph with some bold and some italic'`.	Print 3
426	`Document Title` `A plain paragraph with some bold text and some italic`	`Document Title` `A plain paragraph with some bold and some italic`	Print 3
429	`>>> doc.paragraphs[1].text` `'A plain paragraph with some bold text and some italic'`	`>>> doc.paragraphs[1].text` `'A plain paragraph with some bold and some italic'`	Print 3
442	The code then calls `writerow()` three times to create three **row**, producing a file named *output.tsv* with the following contents:	The code then calls `writerow()` three times to create three **rows**, producing a file named *output.tsv* with the following contents:	Print 3

442	`>>> example_dict_data`
	`[{'Timestamp': '4/5/2035 3:41', 'Fruit': 'Cherries', 'Quantity': '85'},`
		`>>> example_dict_data`
		`[{'Timestamp': '4/5/2035 13:34', 'Fruit': 'Apples', 'Quantity': '73'},`
		`{'Timestamp': '4/5/2035 3:41', 'Fruit': 'Cherries', 'Quantity': '85'},`
			Print 3

445	The `os.makedirs()` call **create** a `headerRemoved` folder in which to save the headless CSV files.	The `os.makedirs()` call **creates** a `headerRemoved` folder in which to save the headless CSV files.	Print 3
451	Its syntax is similar to HTML, which we covered in Chapter **18**, and involves nesting opening and closing tags inside angle brackets that contain other content.	Its syntax is similar to HTML, which we covered in Chapter **13**, and involves nesting opening and closing tags inside angle brackets that contain other content.	Print 3

Page	Error	Correction	Print corrected
463	`print('Lap #{lap_number}: {total_time}({lap_time})', end='')`	`print(f'Lap #{lap_number}: {total_time}({lap_time})', end='')`	Print 2
482	`"<GmailThread len=2 snippet= Do you want to watch RoboCop this weekend?'>"`	`"<GmailThread len=2 snippet='Do you want to watch RoboCop this weekend?'>"`	Print 3
513	Each point is a connecting **points** on the lines you're drawing.	Each point is a connecting **point** on the lines you're drawing.	Print 3
572	The options dictionary passed to `yt_dlp.Youtube()` tells it to download the video . . .	The options dictionary passed to `yt_dlp.YoutubeDL()` tells it to download the video . . .	Print 3
579	`matplotlib==3.92`	`matplotlib==3.9.2`	Print 3